OBJECT ORIENTED PROGRAMMING

TABLE OF CONTENTS

01	Abstract	02
02	Problem Statement	03
03	Introduction	04
04	Methodology	06
05	Flowchart	10
06	Use of OOPS	11
07	Results	12
08	Discussion	13
09	Conclusion	14

ABSTRACT

This project presents a Smart Traffic Management System built using object-oriented programming in C++. The system offers user authentication, traffic signal control based on real-time vehicle count, and emergency handling with higher priority. It includes modules for registering and logging in users, dynamically controlling traffic signals based on vehicle flow and emergency presence, and reporting as well as viewing traffic violations and emergency events. All data is stored and managed through file handling to ensure persistent record-keeping.

A unique challan number is generated for each violation with predefined penalties, while emergencies are categorized (e.g., road accidents, tree/pole falling, bridge collapse) and stored by type and location. The signal control module simulates real-time delays and logs each traffic control instance. This system aims to replicate a simplified yet intelligent traffic control infrastructure ideal for educational, prototyping, or smart city simulation purposes.

PROBLEM STATEMENT

➢ Background

With the exponential rise in urban populations and the growing number of vehicles on the road, city traffic systems are under immense pressure. Traffic congestion, violation of road safety rules, delayed emergency responses, and inefficient traffic signal management have become everyday issues in metropolitan and semi-urban areas. Traditional traffic systems rely heavily on static signal timers and manual law enforcement, which are neither scalable nor responsive to real-time conditions.

Furthermore, manual tracking of traffic violations and emergency incidents often results in incomplete records and ineffective penalization. A system capable of dynamically adjusting to traffic flow, managing emergency priorities, and maintaining accurate logs is essential for ensuring public safety and optimal road utilization.

➢ Problem description

Modern cities face a lack of intelligent, automated, and integrated traffic management systems that can:

- Adjust signal durations based on real-time traffic flow.
- Prioritize emergency vehicles across all directions.
- Log and manage traffic violations with predefined penalty rules.
- Record emergency events with categorization by location and type.
- Ensure persistent, secure data storage without reliance on a live backend.

INTRODUCTION

➢ Overview of the Project :

Urbanization and rapid growth in the number of vehicles have led to increasingly congested roads and overwhelmed traffic management systems. Traditional traffic systems operate on fixed-timer signals and rely heavily on manual enforcement, making them inefficient in handling real-time traffic flow, violations, and emergencies. With smart cities on the rise, there is a pressing need to implement intelligent and adaptable systems that can respond dynamically to real-world traffic conditions.

The **Smart Traffic Management System**, developed using C++ and object-oriented programming, is a simulation-based approach to managing traffic signals, logging violations, and handling emergencies efficiently. This project integrates fundamental system features like user authentication, traffic prioritization, penalty automation, and emergency reporting, while ensuring all records are persistently stored using file handling.

➢ Purpose :

The primary purpose of this project is to design and simulate a smart, efficient, and user-friendly traffic management system that:

- Dynamically adjusts signal durations based on vehicle count.
- Prioritizes directions with emergency vehicles.
- Enables users to report traffic violations with automated challan generation.
- Records and categorizes emergencies by type and location.
- Allows users to log in and interact securely through a menu-driven interface.
- Stores all data persistently for review, statistics, or reporting.

This project is a functional prototype aimed at demonstrating how intelligent traffic systems can be simulated and studied using C++ programming, file handling, and system logic design.

➢ Scope :

This system focuses on simulating a four-directional intersection where:

- Signals are controlled based on vehicle density and emergency priority.
- Emergency vehicles (e.g., ambulances, fire trucks) can override regular signal sequences.
- Violations are reported with specific predefined types such as overspeeding, no helmet, mobile phone use, etc.
- Emergencies like road accidents, bridge collapses, and water logging are reported with location data.
- The system is console-based and file-driven, allowing persistent storage without external databases.

While this version is designed as a simulation, it lays the foundation for integration with real-world hardware such as traffic cameras, IoT sensors, or cloud-based traffic analytics platforms in future expansions.

METHODOLOGY

1. System Overview and Architecture :

The system follows an object-oriented approach, dividing responsibilities among four classes:

- **User** : Handles user registration and login.
- **Traffic Control** : Manages signal timing based on traffic density and emergency presence.
- **Violation**: Records rule violations and generates challans with penalties.
- **Emergency**: Logs street emergencies and retrieves reports based on location/type.

Each class encapsulates relevant data and behavior, allowing for modular expansion and maintenance**.**

2. User Management System :

This module restricts system access to authorized users. It ensures only registered users can operate the traffic control and reporting systems.

Functionality:

- Registration prompts users to create a username and password. These are saved to users.txt.
- Login verifies credentials by scanning the file line-by-line.

```
void registerUser()
{
    cout << "Enter new username: ";
    cin.ignore();
    getline(cin,username);
    cout << "Enter new password: ";
    getline(cin,password);
    ofstream file("users.txt", ios::app);
    file << username << " " << password << endl;
    file.close();
    cout << "Registration successful.\n";
}
```

3. Traffic Signal Control Logic :

It has a smart traffic light controller that dynamically adjusts green signal duration based on:

- Number of waiting vehicles.
- Presence of emergency vehicles like ambulances or fire trucks

Functionality:

- Users input vehicle counts and emergency presence for North, East, South, and West.
- Direction with emergency vehicles are given first priority.
- Directions are sorted based on emergency priority and traffic density.
- The final order is logged in signal_log.txt.

```
    d.greenDuration = d.hasEmergency ? 20 : (20 + d.vehicleCount); // emergency gets 20s
    data.push_back(d);
}
sort(data.begin(), data.end(), [](const DirectionData &a, const DirectionData &b)
        {
if (a.hasEmergency != b.hasEmergency)
        return a.hasEmergency > b.hasEmergency;
return a.vehicleCount > b.vehicleCount; });
```

4. Violation Detection and Challan Generation :

Allows traffic officers (or users) to report violations and automatically calculate the fine (penalty).

Functionality:

- Users choose from predefined violations (e.g., speeding, no helmet).
- Penalties are fixed and mapped to each violation type.
- A unique challan number is generated using a counter stored in challan_id.txt.

- All violation reports are saved in violations.txt.

```
ofstream file("violations.txt", ios::app);
file << "Challan No: " << challanId
     << " | Vehicle: " << vehicleNumber
     << " | Violation: " << violation
     << " | Penalty: Rs" << penalty << "\n";
file.close();
```

5. Emergency Reporting Module :

Simulates citizen or officer reports about road emergencies that could affect traffic.

Functionality:

- Users provide the street name and select the emergency type.
- Data is stored in emergencies.txt.
- Users can view reports filtered by street or emergency.

```
ofstream file("emergencies.txt", ios::app);
file << "Street: " << streetName
     << " | Emergency: " << selectedEmergency << "\n";
file.close();
```

6. Main Control Flow :

The application uses a **main menu loop** where the user chooses to register, log in, or exit. After login, another menu allows users to:

- Control traffic signals
- Report/view violations
- Report/view emergencies

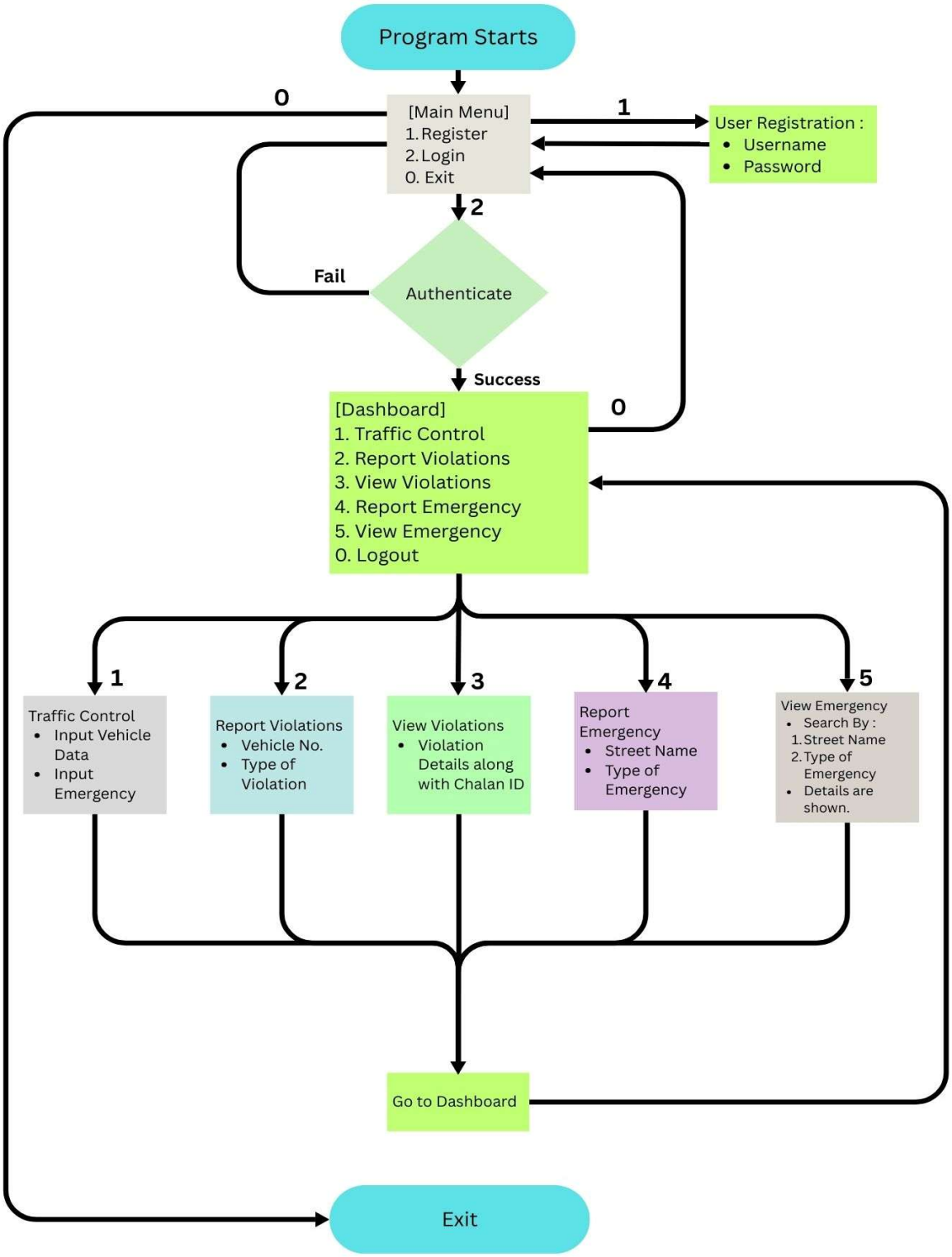
```
if (user.loginUser())
{
    while (true)
    {
        cout << "\n1. Control Traffic Signal\n"
                "2. Report Violation\n"
                "3. View Violations\n"
                "4. Report Emergency\n"
                "5. View Emergencies\n"
                "0. Logout\nChoice: ";
```

7. File Handling for Data Persistence :

Instead of databases, this project uses **text files** to simulate persistent storage:

File Name	Purpose
`users.txt`	Stores usernames and passwords
`violations.txt`	Logs traffic rule violations
`challan_id.txt`	Maintains a counter for challan numbers
`emergencies.txt`	Stores emergency incident logs
`signal_log.txt`	Logs traffic signal decisions

FLOWCHART


Program Starts
[Main Menu]
1. Register
2. Login
0. Exit
0
1
User Registration :
• Username
• Password
2
Fail
Authenticate
Success
[Dashboard]
1. Traffic Control
2. Report Violations
3. View Violations
4. Report Emergency
5. View Emergency
0. Logout
0
1
2
3
4
5
Traffic Control
• Input Vehicle Data
• Input Emergency
Report Violations
• Vehicle No.
• Type of Violation
View Violations
• Violation Details along with Chalan ID
Report Emergency
• Street Name
• Type of Emergency
View Emergency
• Search By :
1. Street Name
2. Type of Emergency
• Details are shown.
Go to Dashboard
Exit

USE OF OOPS

1. Encapsulation :

Encapsulation is the concept of binding data and the methods that operate on that data into a single unit (class) and restricting direct access to some components.

In Our Code:

Each class (User, Traffic Control, Violation, and Emergency) encapsulates its data and functionality. For instance, in the User class, the username and password are private, and access is controlled via public methods.

```
class User
{
  string username, password;
public:
    void registerUser() {}
    bool loginUser()
```

2. Abstraction :

Abstraction is the process of **hiding complex implementation details** and showing only the **essential features** of the object.

In Our Code:

The main function interacts with high-level methods like tc.controlSignal() or violation.reportViolation() without worrying about how the internal logic works.

```
case 1:
    tc.controlSignal(); // abstracted signal control logic

    // Internally it performs complex sorting, duration calculations, emergency handling
    break;
```

3. Inheritance :

Inheritance allows one class to inherit the properties and methods of another, **promoting code reuse.**

4. Polymorphism :

Polymorphism allows objects of **different types to be treated through a common interface**, typically using inheritance + virtual functions.

RESULT

The developed Traffic Management System is a console-based C++ application that simulates and manages various aspects of urban traffic control. The system allows users to:

- Register and log in with a username and password.
- Control traffic signals dynamically based on vehicle count and presence of emergency vehicles.
- Report and view traffic violations with automatic challan generation.
- Report and view emergency cases such as accidents, bridge collapses, etc.

Key Outputs:

- Signal light order with dynamic green light duration based on traffic conditions.
- Penalty challans with unique IDs for each traffic violation.
- Emergency logs searchable by location or type.
- All logs persist in .txt files for records and further processing.

Sample Output :

```
Traffic Signal Green Light Order:
1. East | Vehicles: 40 | Emergency: Yes | Duration: 20 seconds
2. North | Vehicles: 30 | Emergency: No | Duration: 50 seconds
...
Violation reported successfully.
Challan Number: 1003 | Penalty: Rs1200
```

DISCUSSION

The Traffic Management System is designed to mimic a real-world city traffic control system using C++. It supports multiple functional modules, each encapsulated within its own class:

- **User Authentication:**
  - Handles user registration and login with basic username-password authentication.
  - Stores user credentials in users.txt, simulating account management.
- **Traffic Signal Control:**
  - Uses user input (vehicle count and emergency presence) to determine the order and duration of green lights.
  - Gives emergency vehicles priority.
  - Vehicle count influences signal timing, supporting dynamic traffic flow optimization.
  - Logs each control session to signal_log.txt.
- **Violation Reporting:**
  - Users can report traffic violations from a predefined list (e.g., No Helmet, Overspeeding).
  - Automatically assigns penalties and generates a unique Challan ID stored in challan_id.txt.

    - All violation records are saved in violations.txt.
- **Emergency Reporting:**
    - Reports events like accidents or bridge collapses by taking the street name and type of emergency.
    - Stores reports in emergencies.txt.
    - Includes a search feature to filter reports by street or emergency type.

These modules together simulate a lightweight city-level traffic operations center, usable for both educational and prototyping purposes.

CONCLUSION

The C++ Traffic Management System demonstrates a practical application of Object-Oriented Programming principles, especially **Encapsulation** and **Abstraction**. The code successfully mimics real-life operations like traffic light control, fine collection, and emergency reporting.

This project lays a strong foundation for further development into a more robust and scalable system. By adding **Inheritance** and **Polymorphism**, the code can be made more modular and extensible. Integrating a database and graphical interface could elevate this console-based prototype into a fully-functional city-wide traffic management tool.

Future Enhancements:

- Introduce Admin/User role-based access control.
- Implement database integration for better scalability.
- Use inheritance to handle all report types uniformly.
- Integrate with a real-time map API (for future smart cities projects).